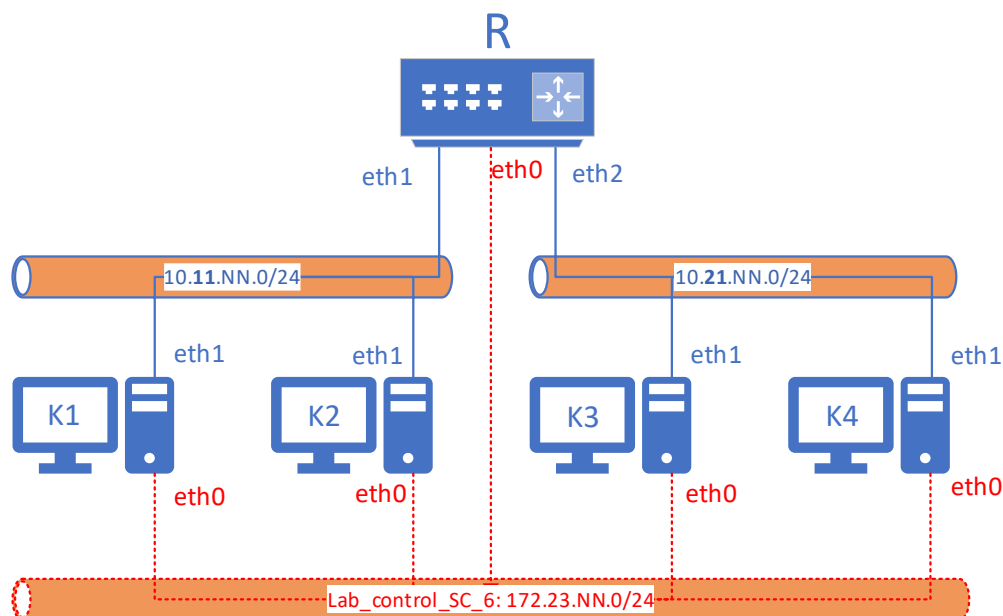


1 Cel ćwiczenia

Celem ćwiczenia jest zapoznanie studentów z podstawami adresacji oraz działania IPv6.

2 Środowisko laboratoryjne

W celu przeprowadzenia laboratorium w trybie zdalnym, dla każdej grupy przygotowano środowisko, które składa się z 5 kontenerów Docker: K1, K2, K3, K4, R, tak jak przedstawiono na rysunku 1.



K1 oraz K2 to dwa „Komputery” wpięte do sieci lokalnej o adresie 10.11.NN.0/24.

K3 oraz K4 to dwa „Komputery” wpięte do sieci lokalnej o adresie 10.21.NN.0/24.

„Ruter” R jest wpięty do obu tych sieci.

Wszystkie kontenery korzystają z systemu Ubuntu Server 20.04 i posiadają doinstalowane dodatkowe paczki programów wykorzystywane w trakcie ćwiczenia.

Wartość NN to numer grupy zadany przez prowadzącego. Oznaczenia grup są dwucyfrowe co ma znaczenie przy wyznaczeniu używanego portu do Logowania (patrz kolejny rozdział). W adresie IP postać 0N zostanie zmieniona na N (np. 10.11.02.0/24 -> 10.11.2.0/24).

Interfejs eth0 służy do podłączenia z siecią zarządzania (dostęp przez SSH / Internet), natomiast interfejsy eth1 i eth2 do przekazu danych między urządzeniami laboratorium. Dodanie interfejsu eth0 pozwala na dowolne manipulowanie interfejsami eth1 i eth2 bez ryzyka zerwania połączenia z kontenerem. **Interfejs eth0 nie powinien być w żaden sposób** (~~ip a a eth0 ...~~) w trakcie ćwiczenia.

Adresy na interfejsach eth1 i eth2 zostały nadane automatycznie przez Docker. **Należy zwrócić uwagę, że pierwszy adres z każdej sieci jest używany przez serwer/host.** Określenie adresów urządzeń jest częścią ćwiczenia.

UWAGA: Prywatny komputer studenta, za pomocą którego wykonywane jest laboratorium, będzie w tej instrukcji nazywany jako: **Komputer Studenta**

2.1 Logowanie

Serwer SSH na kontenerach został wystawiony na konkretny port serwera/hosta o adresie **10.140.0.131**, ale także o adresie IPv6: **fe80::88a1:a5ff:fe5a:a69c/64**, zgodnie z zasadą:

- K1 – 2NN**21**
- K2 – 2NN**22**
- K3 – 2NN**23**
- K4 – 2NN**24**
- R – 2NN**25**

Serwer znajduje się w sieci PL-LAB i powinien być osiągalny po połączeniu do VPN PL-LAB.

Logując się przez SSH należy użyć użytkownika „root” i hasła „rootNN”.

Przykładowo, grupa NN=88 łączy się na Komputer K1 poleceniem:

```
ssh root@10.140.0.131 -p 28821
```

a na Ruter R poleceniem:

```
ssh root@10.140.0.131 -p 28825
```

2.2 Przydatne komendy

2.2.1 Podstawowe komendy

ps -au	wypisanie listy działających procesów
history	wypisuje listę wszystkich wpisanych komend
<tab>	Wciśnięcie klawisza Tabulacji powoduje uzupełnienie wpisywanego tekstu (np. komendy, folderu, czy nazwy pliku)
<Ctrl+c>	Wciśnięcie kombinacji <Ctrl+c> powoduje zatrzymanie działającego procesu
<strzałki>	Strzałki w górę i w dół przewijają ostatnio używane komendy.

Komendy niepotrzebne do wykonania ćwiczenia (ale tak podstawowe, że musiały się tu znaleźć):	
ls	wypisanie zawartości katalogu.
ls -lh	wypisanie zawartości katalogu w postaci listy wraz z właściwościami plików
pwd	Wypisanie ścieżki bezwzględnej katalogu, w którym aktualnie się znajdujemy
cat <plik>	wypisanie zawartości pliku <plik>
cp <plik1> <plik2>	Skopiowanie <plik1> do <plik2>
cd <path>	przejsięcie do katalogu opisanego w ścieżce <path>, ścieżka może być bezwzględna (rozpoczęta znakiem „/”) lub względna. • ścieżka bezwzględna to cała ścieżka, od poziomu „/”, np. komenda <code>cd /root/my_data</code> spowoduje przejście do katalogu <code>/root/my_data</code>, niezależnie od tego, w jakim katalogu komenda została wykonana. • Ścieżka względna to ścieżka wyznaczona od poziomu katalogu, w którym aktualnie się znajdujemy, np. <code>cd my_data</code> wywołane w katalogu <code>/root/</code> spowoduje przejście do katalogu <code>/root/my_data</code>, ale wywołana w innym katalogu zwróci błąd „No such file or directory”. <code>cd ..</code> - przejście poziom wyżej w strukturze katalogów

2.2.2 ping

Program „ping” pozwala na wysłanie zapytania ICMP Echo Request na podany adres IP. Jeśli zapytanie zostanie odebrane przez wywoływany maszynę, powinna ona odpowiedzieć przy pomocy wiadomości ICMP Echo Reply.

- `ping <adres_IP>` - wysłanie pingu na <adres_IP>
- w przypadku pingowania adresu ipv6 typu *link-local* trzeba podać flagę `-I` wraz z nazwą interfejsu, przez który chcemy wysłać ping, np. `ping -I eth1 <adres_IP>`

W poleceniu ping nie podajemy maski adresu IP.

2.2.3 ip a (ip address)

Polecenia z rodziny `ip` zastępują przestarzałe `ifconfig`. Służą do przeglądania i modyfikowania adresacji oraz stanu interfejsów sieciowych. W systemie Windows analogiczna komenda do podglądu to `ipconfig`.

- `ip a (ddress)` - wypisuje informacje o adresach na wszystkich interfejsach
- `ip -br a` - tylko najważniejsze informacje
- `ip -4 a, ip -6 a` - tylko IPv4 lub IPv6
- `ip a s(how) dev <interfejs>` - informacje o danym interfejsie
- `ip a a(dd) <adres>/<maska> dev <interfejs>` - nadanie adresu IP na interfejsie.
Podajemy maskę w notacji prefixu, np. `/24`.
 - o Przykład: `ip a a 10.11.12.13/24 dev eth1`
- `ip a d(el) <adres>/<maska> dev <interfejs>` - usunięcie adresu z interfejsu.
- `ip a flush dev <interfejs>` - wyczyszczenie wszystkich adresów z interfejsu
- `ip link set <interfejs> up / down` - włączenie/wyłączenie interfejsu
- Analogiczne komendy stosujemy dla konfiguracji IPv6, po dodaniu flagi `-6`, np. :
 - o `ip -6 addr add 2001:db8:1234:19::10/64 dev eth1`

2.2.4 ip r (ip route)

`ip route` służy do manipulowania tablicą routingu danej maszyny (wybór trasy, bramki, metryki). Wpisy w tej tablicy są dodawane automatycznie na podstawie adresów nadanych interfejsom, wynikają z konfiguracji, lub mogą być dodawane ręcznie. W systemie Windows odpowiednik podglądu to `route print` (dla IPv6: `route -6 print`).

- `ip r (oute)` - wypisuje tablicę routingu
- `route -6 -n` - stara wersja wypisywania tablicy – dużo czytelniejsza
- `ip -4 r, ip -6 r` - tylko IPv4 / IPv6
- `ip r a(dd) <sieć>/<maska> via <adres_bramy>` - dodanie trasy do sieci przez bramę.
 - o Przykład: `ip r a 10.11.13.0/24 via 10.11.12.1`
- `ip r a default via <adres_bramy> dev <interfejs>` - ustawienie trasy domyślnej (*default gateway*):
 - o Przykład: `ip r a default via 10.11.12.1`
- `ip r d(el) <sieć>/<maska> via <adres_bramy>` - usunięcie wpisu z tablicy routingu.
- Analogiczne komendy stosujemy dla konfiguracji IPv6, po dodaniu flagi `-6`, np. :
 - o `ip -6 r a 2001::/64 via 2002:db8::1 dev eth1`

2.2.5 tcpdump

Polecenie „tcpdump” służy do obserwacji ruchu na interfejsie sieciowym.

- `tcpdump -i <interfejs>` - przechwytuje ramki odbierane na <interfejsie> i wyświetla na ekran podstawowe informacje o tych ramach
 - dodanie flagi `-n` - bez nazw (preferowane)
 - dodanie flagi `-v` - wyświetla bardziej szczegółowe dane (czasami przydatne)
 - flagi można łączyć
- Jeśli szukamy jakiegoś konkretnego ruchu, to warto stosować filtry. Można filtrować pod kątem różnych parametrów, można łączyć filtry, można stosować zaprzeczenia, np.:
 - `tcpdump -i eth0 -n port 80`
 - `tcpdump -i eth0 -n arp`
 - `tcpdump -i eth0 -n icmp and host 10.1.1.1`
 - `tcpdump -i eth0 -n host 10.1.1.1 and not port 22`
 - W szczególności filtr `ip6` – tylko ruch IPv6
 - Ściąga filtrów: <https://www.tcpdump.org/manpages/pcap-filter.7.html>

2.2.6 ss

Program „ss” (z pakietu *iproute2*) wyświetla listę gniazd/połączeń sieciowych i zastępuje przestarzałe `netstat`. Warto zapamiętać zestaw flag „tulpn” (np. jako „tulipan”).

- `ss -tulpn` – pokazuje **tylko nasłuchujące** (`-l`) gniazda TCP/UDP (`-t`, `-u`) wraz z PID/programem (`-p`); adresy i porty w formie numerycznej (`-n`).
- `ss -tupn` – pokazuje **tylko aktywne** gniazda (tj. istniejące połączenia) TCP/UDP (`-t`, `-u`) wraz z PID/programem (`-p`); adresy i porty w formie numerycznej (`-n`).
- `ss -tupna` – pokazuje **wszystkie** gniazda TCP/UDP ...

2.2.7 nslookup

Polecenie to może być użyte w systemie Windows jak i Linux do wyszukiwania szczegółowych informacji odnoszących się do serwerów DNS włączając adres IP poszczególnych komputerów, nazwę domeny, czy aliasy jakie posiada. Nazwa oznacza z angielskiego *name server lookup*. [za wikipedią]

- `nslookup <nazwa_domenowa>` - wyświetla adresy IP pod jakimi dostępna jest dana nazwa_domenowa (np. onet.pl, facebook.com itd.)

3 Zadania do wykonania

W tej części przedstawione zostaną zadania do wykonania. Wykonanie poszczególnych zadań należy potwierdzać odpowiednim zrzutem ekranu, lub opisem otrzymanego wyniku (zgodnie z poleceniem).

3.1 Zespół projektowy

Imię	nazwisko	nr indeksu	adres otrzymany z VPN
Tymon	Zadara	337086	10.141.6.16
Jan	Czechowski	337066	
Kinga	Konieczna	337072	

3.2 Połączenie z siecią PL-LAB

W celu zapewnienia połączenia z siecią laboratoryjną wykorzystano dostęp przez bramkę VPN. Po podłączeniu do bramki VPN, Komputerowi Studenta powinien zostać przyznany adres z puli 10.141.6.0-10.141.6.255/9¹, który umożliwi dostęp do sieci 10.128.0.0/9.

3.2.1 Weryfikacja otrzymanych adresów i tablicy routingu

Proszę zamieścić zrzuty ekranu z terminala **Komputera Studenta**, które potwierdzają otrzymanie adresu z puli 10.141.6.0-10.141.6.255 oraz wpisu do tablicy routingu do sieci 10.128.0.0/9. Należy udokumentować czy został nadany adres IPv6.

W przypadku większej ilości tekstu na zrzucie ekranu, proszę zaznaczyć fragment bezpośrednio odnoszący się do polecenia.

Zrzut ekranu – adres IPv4 i IPv6

Został nadany adres IPv6.

```
Unknown adapter OpenVPN TAP-Windows6:
```

```
Connection-specific DNS Suffix . :
```

```
Link-local IPv6 Address . . . . . : fe80::e86b:1677:3781:5313%3
```

```
IPv4 Address. . . . . : 10.141.6.16
```

```
Subnet Mask . . . . . : 255.128.0.0
```

```
Default Gateway . . . . . :
```

Zrzut ekranu – tablica routingu

¹ Serwer VPN będzie przydzielać adresy z puli 10.141.6.1-10.141.6.255, ale maska /9 zapewnia dostęp do sieci 10.128.0.0/9, tj. do adresów z puli 10.128.0.0 – 10.255.255.255.

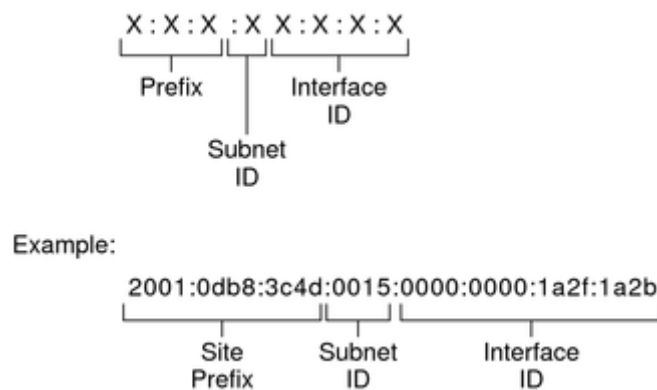
IPv4 Route Table				
=====				
Active Routes:				
Network	Destination	Netmask	Gateway	Interface Metric
0.0.0.0	0.0.0.0	0.0.0.0	192.168.63.254	192.168.61.76 35
10.128.0.0	255.128.0.0		On-link	10.141.6.16 281
10.141.6.16	255.255.255.255		On-link	10.141.6.16 281
10.255.255.255	255.255.255.255		On-link	10.141.6.16 281
127.0.0.0	255.0.0.0		On-link	127.0.0.1 331
127.0.0.1	255.255.255.255		On-link	127.0.0.1 331
127.255.255.255	255.255.255.255		On-link	127.0.0.1 331
192.168.60.0	255.255.252.0		On-link	192.168.61.76 291
192.168.61.76	255.255.255.255		On-link	192.168.61.76 291
192.168.63.255	255.255.255.255		On-link	192.168.61.76 291
194.29.131.177	255.255.255.255		10.140.0.121	10.141.6.16 281
194.29.160.103	255.255.255.255		10.140.0.121	10.141.6.16 281
194.29.169.129	255.255.255.255		10.140.0.121	10.141.6.16 281
224.0.0.0	240.0.0.0		On-link	127.0.0.1 331
224.0.0.0	240.0.0.0		On-link	10.141.6.16 281
224.0.0.0	240.0.0.0		On-link	192.168.61.76 291
255.255.255.255	255.255.255.255		On-link	127.0.0.1 331
255.255.255.255	255.255.255.255		On-link	10.141.6.16 281
255.255.255.255	255.255.255.255		On-link	192.168.61.76 291

Zrzut ekranu – tablica routingu IPv6

IPv6 Route Table			
=====			
Active Routes:			
If	Metric	Network Destination	Gateway
1	331	::1/128	On-link
3	281	fe80::/64	On-link
4	291	fe80::/64	On-link
4	291	fe80::8cd5:caf8:26af:dc00/128	On-link
3	281	fe80::e86b:1677:3781:5313/128	On-link
1	331	ff00::/8	On-link
3	281	ff00::/8	On-link
4	291	ff00::/8	On-link

Adres otrzymany z VPN może się zmieniać po restarcie połączenia. To nie jest problem - może pojawić się jedynie rozbieżność w zadaniu 3.5.1.1. Otrzymany adres można zobaczyć przy pomocy opcji „pokaż status” programu OpenVPN, lub bezpośrednio w terminalu swojego komputera.

Adres IPv6 ma następującą strukturę:



Adres IPv6 podzielony jest na 8 szesnastobitowych bloków zapisanych przy użyciu notacji heksadecymalnej. Aby zaznaczyć, do jakiej sieci należy dany adres, adres ten można podać także w zapisie z długością prefixu (prefix length, lub w skrócie „prefix²”) występującą po ukośniku. np. sieć 2000::/64 to

2000:0000:0000:0000:0000:0000:0000:0000/64 z czego pierwsze 64 bity, czyli

2000:0000:0000:0000 są wspólne dla wszystkich urządzeń, tym samym pierwszy adres z tej sieci, to:

2000:0000:0000:0000:0000:0000:0000:0000/64, a ostatni z tej sieci:

2000:0000:0000:0000:ffff:ffff:ffff:ffff/64

Jeden nieprzerwany ciąg zer można zamienić w „:”, więc np.:

2000:0000:0000:0000:0000:0000:0000:0000/64 = 2000::/64

2000:0000:0000:0000:0000:0000:0023:0010/64 = 2000::23:0010/64

Zakres adresów z sieci 2000::/64 rozciąga się więc między

2000:: a 2000::ffff:ffff:ffff:ffff

Analogicznie zakres adresów w sieci 2000::/32 rozciąga się między

2000:: a 2000::ffff:ffff:ffff:ffff:ffff:ffff

3.2.1.1 Iloma bitami jest zapisana pojedyncza wartość heksadecymalna (0-F) z danego bloku?

Odp: Pojedyncza wartość heksadecymalna z danego bloku jest zapisana 4-ema bitami.

3.2.1.2 Iloma pojedynczymi wartościami heksadecymalnymi jest zapisany jeden bajt?

Odp:: Jeden bajt jest zapisany 2-oma pojedynczymi wartościami heksadecymalnymi

² Używanie pojęcia „prefix” może powodować pewne problemy z niejednoznacznością, ponieważ mamy „prefix site” i „prefix length”, które znaczą co innego – wartość prefixu lub długość prefixu.

3.2.2 Połączenie na kontenery oraz między kontenerami

Należy połączyć się na kontenery K1, R, K3

3.2.2.1 Należy sprawdzić adres MAC, adres IPv4 oraz adres IPv6 interfejsów eth0, eth1, eth2 na K1, R i K3

K1	Adres MAC	Adres IPv4	Adres IPv6
eth0:	0e:1f:b6:b9:14:63	172.23.7.6/24	fe80::c1f:b6ff:feb9:1463/64
eth1:	56:cd:10:3f:6a:5c	10.11.7.3/24	fe80::54cd:10ff:fe3f:6a5c/64

R	Adres MAC	Adres IPv4	Adres IPv6
eth0:	7a:75:ab:78:79:8e	172.23.7.4/24	f380::7875:abff:fe78:798e/64
eth1:	3a:a9:03:fe:31:3a	10.11.7.254/24	fe80::38a9:3ff:fefe:313a/64
eth2:	c2:d7:8c:99:28:bc	10.21.7.254/24	fe80::c0d7:8cff:fe99:28bc/64

K3	Adres MAC	Adres IPv4	Adres IPv6
eth0:	1e:15:ef:99:fb:6d	172.23.7.3/24	fe80::1c15:efff:fe99:fb6d/64
eth1:	da:c8:aa:5c:6f:00	10.21.7.3/24	fe80::d8c8:aaff:fe5c:6f00/64

R

```
root@07_R:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if1271: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 7a:75:ab:78:79:8e brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.23.7.4/24 brd 172.23.7.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::7875:abff:fe78:798e/64 scope link
        valid_lft forever preferred_lft forever
3: eth1@if1275: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 3a:a9:03:fe:31:3a brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.11.7.254/24 brd 10.11.7.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::38a9:3ff:fefe:313a/64 scope link
        valid_lft forever preferred_lft forever
4: eth2@if1277: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether c2:d7:8c:99:28:bc brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.21.7.254/24 brd 10.21.7.255 scope global eth2
        valid_lft forever preferred_lft forever
    inet6 fe80::c0d7:8cff:fe99:28bc/64 scope link
        valid_lft forever preferred_lft forever
```

K1

```
root@07_K1:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth1@if1270: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 56:cd:10:3f:6a:5c brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.11.7.3/24 brd 10.11.7.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::54cd:10ff:fe3f:6a5c/64 scope link
        valid_lft forever preferred_lft forever
3: eth0@if1276: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 0e:1f:b6:b9:14:63 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.23.7.6/24 brd 172.23.7.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::c1f:b6ff:feb9:1463/64 scope link
        valid_lft forever preferred_lft forever
```

K2

```
root@07_K3:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if1269: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 1e:15:ef:99:fb:6d brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.23.7.3/24 brd 172.23.7.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::1c15:efff:fe99:fb6d/64 scope link
        valid_lft forever preferred_lft forever
3: eth1@if1274: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether da:c8:aa:5c:6f:00 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.21.7.3/24 brd 10.21.7.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::d8c8:aaff:fe5c:6f00/64 scope link
        valid_lft forever preferred_lft forever
```

3.2.2.2 Dlaczego wszystkie powyższe adresy IPv6 zaczynają się od FE80?

Odp: Prefix FE80 to specjalnie zarezerwowany prefix dla adresów wygenerowanych lokalnie. Wszystkie te adresy to są adresy wygenerowane dla sieci lokalnej i dlatego zaczynają się od FE80.

3.2.2.3 Dlaczego w obserwowanych adresach IPv6 (na K1, R, K3) nie widać pełnych 8 bloków adresu?

Odp: Część adresów które składają się z bloków czterech zer ('0000') zostają zamienione na ::.

przykład:

f380::7875:abff:fe78:798e/64 – oryginalnie to f380:0000:0000:0000:7875:abff:fe78:798e/64 ale dla przejrzystości ciąg bloków '0000' zostaje zamieniony na '::'. (uwaga! Tylko jeden ciąg bloków '0000' może być zamieniony na '::')

3.2.2.4 Czy któryś z obserwowanych adresów IPv6 posiada blok zapisany inną liczbą wartości heksadecymalnych niż 4 lub 0? Jeśli tak, proszę ten adres przytoczyć i wytłumaczyć dlaczego dany blok reprezentowany jest jednym, dwoma, lub trzema znakami wartościami hex.

Odp: Tak. Adres fe80::38a9:3ff:fefe:313a/64 posiada blok zapisany trzema wartościami heksadecymalnymi – 3ff. Ten blok jest równoznaczny z zapisem 03ff, natomiast w celach optymalizacyjnych, w IPv6 można pomijać w zapisie początkowe zera w każdym bloku.

3.2.2.5 Jaka zasada łączy adres MAC oraz adres IPv6 danego urządzenia w przypadku autokonfiguracji adresu IPv6 (SLAAC)? Zaprezentuj na przykładzie wybranej pary z tabelki 3.2.2.1

Odp:

Adres MAC identyfikuje urządzenie fizyczne (kartę sieciową, interfejs), nazywamy go też adresem sprzętowym. Adres MAC składa się z 6 bajtów (48 bitów). Przykładowy adres MAC to np. **00:0A:E6:3E:FD:E1**. Pierwsze 3 bajty określają producenta, kolejne 3 - numer egzemplarza karty.

Należy spróbować spingować K3 z K1 używając jako adresu docelowego

- a) adresu K3/eth0/IPv6*
- b) adresu K3/eth1/IPv6

Uwaga: Pingowany adres jest adresem typu *link-local*, więc trzeba podać, przez który interfejs chcemy wysłać ping (tj. `ping -I ethX <adres>`)

**Przez adres „K3/eth0/IPv6” należy rozumieć adres IPv6 na interfejsie eth0 komputera K3, przez adres „K3/eth0” należy rozumieć adres IPv4 na interfejsie eth0 komputera K3.*

3.2.2.6 Jaki jest wynik działania polecenia ping w obu przypadkach?

Ping na adres K3/eth0/IPv6 - Zrzut ekranu z K1

```
root@07_K1:~# ping -I eth0 fe80::1c15:efff:fe99:fb6d
ping: Warning: source address might be selected on device other than: eth0
PING fe80::1c15:efff:fe99:fb6d(fe80::1c15:efff:fe99:fb6d) from :: eth0: 56 data bytes
64 bytes from fe80::1c15:efff:fe99:fb6d%eth0: icmp_seq=1 ttl=64 time=0.348 ms
64 bytes from fe80::1c15:efff:fe99:fb6d%eth0: icmp_seq=2 ttl=64 time=0.043 ms
64 bytes from fe80::1c15:efff:fe99:fb6d%eth0: icmp_seq=3 ttl=64 time=0.043 ms
64 bytes from fe80::1c15:efff:fe99:fb6d%eth0: icmp_seq=4 ttl=64 time=0.043 ms
64 bytes from fe80::1c15:efff:fe99:fb6d%eth0: icmp_seq=5 ttl=64 time=0.044 ms
64 bytes from fe80::1c15:efff:fe99:fb6d%eth0: icmp_seq=6 ttl=64 time=0.044 ms
```

Ping na adres K3/eth1/IPv6 - Zrzut ekranu z K1

```
root@07_K1:~# ping -I eth1 fe80::d8c8:aaff:fe5c:6f00
ping: Warning: source address might be selected on device other than: eth1
PING fe80::d8c8:aaff:fe5c:6f00(fe80::d8c8:aaff:fe5c:6f00) from :: eth1: 56 data bytes
From fe80::54cd:10ff:fe3f:6a5c%eth1 icmp_seq=1 Destination unreachable: Address unreachable
ping: sendmsg: No route to host
From fe80::54cd:10ff:fe3f:6a5c%eth1 icmp_seq=2 Destination unreachable: Address unreachable
From fe80::54cd:10ff:fe3f:6a5c%eth1 icmp_seq=3 Destination unreachable: Address unreachable
From fe80::54cd:10ff:fe3f:6a5c%eth1 icmp_seq=5 Destination unreachable: Address unreachable
ping: sendmsg: No route to host
From fe80::54cd:10ff:fe3f:6a5c%eth1 icmp_seq=6 Destination unreachable: Address unreachable
From fe80::54cd:10ff:fe3f:6a5c%eth1 icmp_seq=7 Destination unreachable: Address unreachable
```

3.2.2.7 Dlaczego otrzymano inny wynik?

Odp: K1 nie ma połączenia obecnie z K3, ponieważ nie jest zdefiniowane w routingu jak K1 ma się połączyć z K3 (są w różnych sieciach lokalnych i aby skomunikować się przez eth1 muszą przejść przez router który nie jest jeszcze skonfigurowany). K1 nie widzi K3 i tym samym nie ma jak zpingować K3.

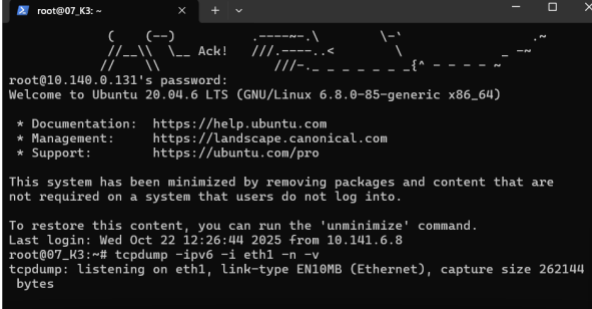
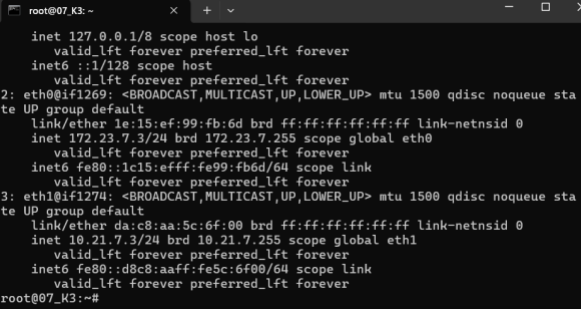
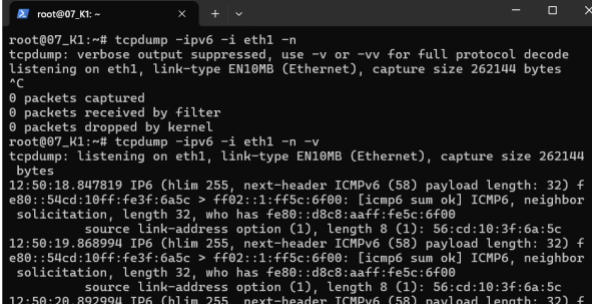
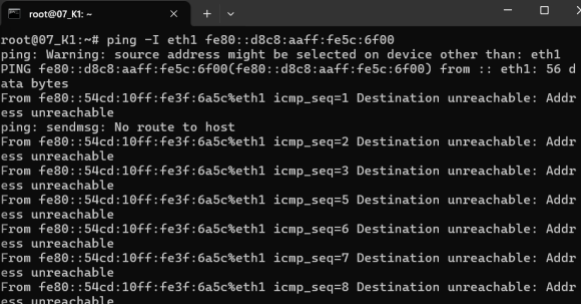
3.3 Test autokonfiguracji IPv6

Z jednego komputera należy mieć uruchomione 2 połączenia SSH na K1 oraz 2 połączenia SSH na K3, tak by jednocześnie widzieć 4 terminale.

Na dwóch terminalach uruchomić obserwowanie ruchu IPv6 na interfejsach **eth1** kontenerów K1 i K3. *Podpowiedź: należy użyć polecenia: `tcpdump -ipv6 -i eth1 -n` (opcjonalnie można dodać flagę `-v`, `-vv`, lub `-vvv`)*

Z kolejnego terminala **uruchomić ping z K1 na K3** (używając jako adresu docelowego adresu **K3/eth1/IPv6**). Uwaga: Pingowany adres jest adresem typu *link-local*, więc trzeba podać, przez który interfejs chcemy wysłać ping (tj. `ping -I ethX <adres>`)

3.3.1.1 Czy jakiś ruch jest widoczny na którymkolwiek z interfejsów?

Zrzut ekranu	4 terminale
	
	

Odp: Widoczny jest ruch na terminalu obserwującym IPv6 na interfejsie eth1 kontenera K1. Na terminalu obserwującym IPv6 na interfejsie eth1 kontenera K3 nie ma żadnego ruchu.

Zatrzymać obserwowanie ruchu na interfejsach **eth1** kontenerów K1 i K3.

Uruchomić obserwowanie ruchu IPv6 na interfejsach **eth0** kontenerów K1 i K3 (`tcpdump`). *Podpowiedź: warto użyć odpowiedniego filtrowania ruchu, bo inaczej będzie widoczne bardzo dużo ruchu SSH.*

3.3.1.2 Czy jakiś ruch jest widoczny na którymkolwiek z interfejsów?

Zrzut ekranu – 4 terminale

Odp: Ruch jest widoczny zarówno na interfejsie kontenera K1, jak i na interfejsie kontenera K3. Pakiety odebrane zgadzają się z pakietami przedstawionymi w odpowiedzi. Żeby pakiety były widoczne interfejs (eth0/eth1) musi być taki sam dla komendy ping jak i komendy tcpdump. W przeciwnym wypadku nie widać ruchu na terminalu.

Zatrzymać program ping (Ctrl+c).

Podpowiedź: powinny zostać odebrane pakiety tego typu (w zależności od tego, czy wysyłałiśmy ruch interfejsem eth0 czy eth1 – będą widoczne na eth0 lub eth1 kontenera K1):

```
14:40:11.997017 IP6 fe80::8ca:89ff:fead:191e > ff02::1:ff60:e081: ICMP6, neighbor solicitation, who has fe80::a4:6dff:fe60:e081, length 32
14:40:13.021024 IP6 fe80::8ca:89ff:fead:191e > ff02::1:ff60:e081: ICMP6, neighbor solicitation, who has fe80::a4:6dff:fe60:e081, length 32
14:40:14.045167 IP6 fe80::8ca:89ff:fead:191e > ff02::1:ff60:e081: ICMP6, neighbor solicitation, who has fe80::a4:6dff:fe60:e081, length 32
14:40:15.069026 IP6 fe80::8ca:89ff:fead:191e > ff02::1:ff60:e081: ICMP6, neighbor solicitation, who has fe80::a4:6dff:fe60:e081, length 32
14:40:16.093027 IP6 fe80::8ca:89ff:fead:191e > ff02::1:ff60:e081: ICMP6, neighbor solicitation, who has fe80::a4:6dff:fe60:e081, length 32
```

3.3.1.3 Wyjaśnić co to za adres ff02::1:ffxx:xxxx (podpowiedź: wyszukaj „neighbor solicitation” i „Solicited-node multicast address”). Jaką funkcję pełni ten adres?

Odp: Adres ff02::1:ffxx:xxxx to Solicited-Node Multicast Address (Adres Multicast Węzła Żądanego), pełniący w protokole IPv6 (NDP) funkcję celu dla wiadomości Neighbor Solicitation (NS). Adres ten pozwala, by pakiety NS docierały wyłącznie do węzła o poszukiwanym adresie.

3.3.1.4 Sprawdzić tablicę routingu IPv6 na K1 (zarówno z flagą -n jak i bez niej).

Zrzut ekranu – tablica routingu IPv6 K1 (bez -n)

```
root@07_K1:~# route -6
Kernel IPv6 routing table
```

Destination	Next Hop	Flag	Met	Ref	Use	If
fe80::/64	:::	U	256	18	0	eth1
fe80::/64	:::	U	256	10	0	eth0
:::/0	s9-3	UG	1024	1	0	eth0
localhost/128	:::	Un	0	8	0	lo
fe80::/128	:::	Un	0	4	0	eth1
fe80::/128	:::	Un	0	3	0	eth0
fe80::c1f:b6ff:feb9:1463/128	:::	Un	0	11	0	eth0
fe80::54cd:10ff:fe3f:6a5c/128	:::	Un	0	19	0	eth1
ip6-mcastprefix/8	:::	U	256	11	0	eth1
ip6-mcastprefix/8	:::	U	256	7	0	eth0
:::/0	:::	!n	-1	1	0	lo

Zrzut ekranu – tablica routingu IPv6 K1 (z -n)

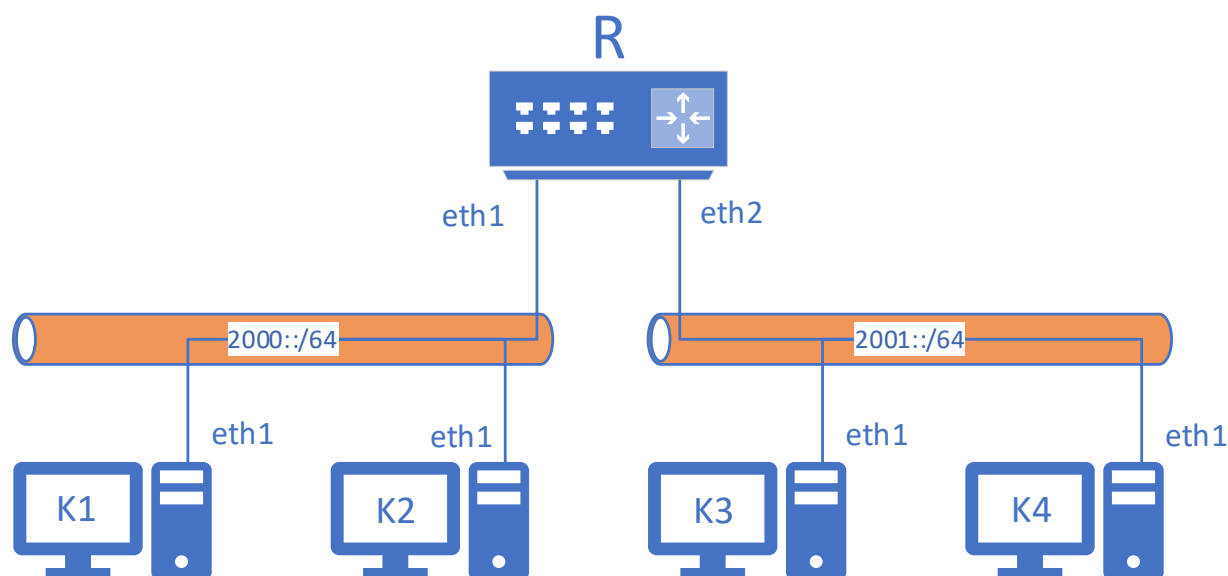
```
root@07_K1:~# route -6 -n
Kernel IPv6 routing table
```

Destination	Next Hop	Flag	Met	Ref	Use	If
fe80::/64	::	U	256	18	0	eth1
fe80::/64	::	U	256	10	0	eth0
::/0	fd37:e7a1:5a65:12::1	UG	1024	1	0	eth0
::1/128	::	Un	0	8	0	lo
fe80::/128	::	Un	0	4	0	eth1
fe80::/128	::	Un	0	3	0	eth0
fe80::c1f:b6ff:feb9:1463/128	::	Un	0	11	0	eth0
fe80::54cd:10ff:fe3f:6a5c/128	::	Un	0	19	0	eth1
ff00::/8	::	U	256	11	0	eth1
ff00::/8	::	U	256	7	0	eth0
::/0	::	!n	-1	1	0	lo

3.3.1.5 Do czego służą wiadomości typu „neighbor solicitation/advertisement”? Czy można nawiązać analogię do jakiegoś mechanizmu znanego z IP (v4)?

Odp: W protokole IPv6 mechanizm **Neighbor Solicitation (NS) / Neighbor Advertisement (NA)** służy do wykrywania sąsiadów w sieci lokalnej oraz ustalania ich adresów MAC. Host wysyła wiadomość NS, pytając „kto ma dany adres IPv6?”, a docelowy host odpowiada NA podając swój adres MAC. Mechanizm ten pełni funkcję analogiczną do protokołu **ARP w IPv4**, ale jest zintegrowany z protokołem ICMPv6 i dodatkowo pozwala sprawdzić dostępność hostów w sieci.

3.4 Podstawowa konfiguracja routingu IPv6



Narzucone wymagania na sieć:

1. K1, K2 i R mają należeć do tej samej sieci IPv6, o adresie 2000::/64
2. K3, K4 i R mają należeć do tej samej sieci IPv6, o adresie 2001::/64
3. Ostatni blok adresu IPv6 dla K1 i K3 wynosi 0001
4. Ostatni blok adresu R dla obu sieci IPv6 wynosi 1234
5. Komputery K1 i K3 komunikują się za pośrednictwem rutera R

Aby oszczędzić pracy pominiemy konfigurację K2 i K4. Poprawne wykonanie zadań w punktach 3.4.1.2 oraz 3.4.1.6 zapewni wymaganą konfigurację sieci.

3.4.1.1 Jakie adresy powinny zostać przypisane do poszczególnych interfejsów, aby wypełnić wymagania (1-4):

	eth1/IPv6	eth2/IPv6
K1	2000::1/64	-brak-
K3	2001::1/64	-brak-
R	2000::1234/64	2001::1234/64

3.4.1.2 Należy ustawić adresy IPv6 na K1, R, K3 zgodnie z powyższą tabelką. Nie usuwaj adresu link-local, a dodaj drugi. Udokumentuj zrzutami ekranu:

```

root@07_K1:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.23.7.6 netmask 255.255.255.0 broadcast 172.23.7.255
    inet6 fe80::c1f:b6ff:feb9:1463 prefixlen 64 scopeid 0x20<link>
    ether 0e:1f:b6:b9:14:63 txqueuelen 0 (Ethernet)
    RX packets 3061 bytes 245761 (245.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2318 bytes 353319 (353.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.11.7.3 netmask 255.255.255.0 broadcast 10.11.7.255
    → inet6 2000::1 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::54cd:10ff:fe3f:6a5c prefixlen 64 scopeid 0x20<link>
    ether 56:cd:10:3f:6a:5c txqueuelen 0 (Ethernet)
    RX packets 30 bytes 3208 (3.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 152 bytes 13340 (13.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

root@07_R:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.23.7.4 netmask 255.255.255.0 broadcast 172.23.7.255
    inet6 fe80::7875:abff:fe78:798e prefixlen 64 scopeid 0x20<link>
    ether 7a:75:ab:78:79:8e txqueuelen 0 (Ethernet)
    RX packets 308 bytes 28465 (28.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 180 bytes 26261 (26.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.11.7.254 netmask 255.255.255.0 broadcast 10.11.7.255
    inet6 fe80::38a9:3ff:fefe:313a prefixlen 64 scopeid 0x20<link>
    → inet6 2000::1234 prefixlen 64 scopeid 0x0<global>
    ether 3a:a9:03:fe:31:3a txqueuelen 0 (Ethernet)
    RX packets 165 bytes 14818 (14.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 15 bytes 1602 (1.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.21.7.254 netmask 255.255.255.0 broadcast 10.21.7.255
    inet6 fe80::c0d7:8cff:fe99:28bc prefixlen 64 scopeid 0x20<link>
    → inet6 2001::1234 prefixlen 64 scopeid 0x0<global>
    ether c2:d7:8c:99:28:bc txqueuelen 0 (Ethernet)
    RX packets 20 bytes 2136 (2.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14 bytes 1492 (1.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

root@07_K3:~# ifconfig
-bash: ifconfig: command not found
root@07_K3:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.23.7.3 netmask 255.255.255.0 broadcast 172.23.7.255
    inet6 fe80::1c15:efff:fe99:fb6d prefixlen 64 scopeid 0x20<link>
    ether 1e:15:ef:99:fb:6d txqueuelen 0 (Ethernet)
    RX packets 970 bytes 85876 (85.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 759 bytes 146807 (146.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.21.7.3 netmask 255.255.255.0 broadcast 10.21.7.255
    → inet6 2001::1 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::d8c8:aaff:fe5c:6f00 prefixlen 64 scopeid 0x20<link>
    ether da:c8:aa:5c:6f:00 txqueuelen 0 (Ethernet)
    RX packets 31 bytes 3406 (3.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 15 bytes 1602 (1.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Zwróć uwagę, że interfejsy eth1 (na R również eth2) powinny mieć ustawione po 2 adresy IPv6 (link-local fe80:... oraz nowo nadany adres). Jeśli został dodany zły adres, można go usunąć (`ip -6 addr del ...`). W dalszej części ćwiczenia korzystamy z tych nowych adresów.

3.4.1.3 Wykonaj ping z K1 na nowo ustawiony adres R/eth1/IPv6 oraz ping z K3 na R/eth2/IPv6.

Podpowiedź: Jeśli adresy są ustawione prawidłowo, to powinno się pingować.

```

root@07_K1:~# ping 2000::1234
PING 2000::1234(2000::1234) 56 data bytes
64 bytes from 2000::1234: icmp_seq=1 ttl=64 time=0.153 ms
64 bytes from 2000::1234: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 2000::1234: icmp_seq=3 ttl=64 time=0.049 ms
64 bytes from 2000::1234: icmp_seq=4 ttl=64 time=0.055 ms
64 bytes from 2000::1234: icmp_seq=5 ttl=64 time=0.042 ms
64 bytes from 2000::1234: icmp_seq=6 ttl=64 time=0.064 ms
64 bytes from 2000::1234: icmp_seq=7 ttl=64 time=0.041 ms
64 bytes from 2000::1234: icmp_seq=8 ttl=64 time=0.041 ms
64 bytes from 2000::1234: icmp_seq=9 ttl=64 time=0.040 ms
64 bytes from 2000::1234: icmp_seq=10 ttl=64 time=0.041 ms
64 bytes from 2000::1234: icmp_seq=11 ttl=64 time=0.031 ms

```

```

root@07_K3:~# ping 2001::1234
PING 2001::1234(2001::1234) 56 data bytes
64 bytes from 2001::1234: icmp_seq=1 ttl=64 time=0.151 ms
64 bytes from 2001::1234: icmp_seq=2 ttl=64 time=0.043 ms
64 bytes from 2001::1234: icmp_seq=3 ttl=64 time=0.043 ms
64 bytes from 2001::1234: icmp_seq=4 ttl=64 time=0.066 ms
64 bytes from 2001::1234: icmp_seq=5 ttl=64 time=0.042 ms
64 bytes from 2001::1234: icmp_seq=6 ttl=64 time=0.043 ms
64 bytes from 2001::1234: icmp_seq=7 ttl=64 time=0.043 ms
64 bytes from 2001::1234: icmp_seq=8 ttl=64 time=0.042 ms
64 bytes from 2001::1234: icmp_seq=9 ttl=64 time=0.043 ms
64 bytes from 2001::1234: icmp_seq=10 ttl=64 time=0.042 ms
64 bytes from 2001::1234: icmp_seq=11 ttl=64 time=0.042 ms
64 bytes from 2001::1234: icmp_seq=12 ttl=64 time=0.058 ms

```

3.4.1.4 Wykonaj ping z K1 na K3/eth1/IPv6. Podpowiedź: nie powinno działać – „ping: connect: Network is unreachable”

```

root@07_K1:~# ping 2001::1
PING 2001::1(2001::1) 56 data bytes

```

Ping z K1 na K3/eth1/IPv6 nie działa. Po wykonaniu komendy ping czekamy na odpowiedź, ale jej nie otrzymujemy (ping wędruje do defaultu czyli na eth0).

Sięgając pamięcią do laboratorium IPv4, spodziewacie się, że na K1 (K3) brakuje wpisu w tablicy routingu, który kierowałby pakiety adresowane do sieci 2001::/64 (2000::/64), na odpowiednią bramę, czyli na odpowiedni interfejs rutera R.

3.4.1.5 Należy sprawdzić tablicę routingu Ipv6 na K1.

```

root@07_K1:~# ip -6 r
2000::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth0 proto kernel metric 256 pref medium
default via fd37:e7a1:5a65:12::1 dev eth0 metric 1024 pref medium

```

3.4.1.6 Należy dodać wpisy do tablicy routingu na K1 i K3 tak, by zapewnić komunikację między tymi komputerami za pośrednictwem R (wymaganie 5 z początku rozdziału). Należy sprawdzić tablicę routingu Ipv6 na K1 i K3

```

root@07_K1:~# ip -6 r
2000::/64 dev eth1 proto kernel metric 256 pref medium
2001::/64 via 2000::1234 dev eth1 metric 1024 pref medium
fe80::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth0 proto kernel metric 256 pref medium
default via fd37:e7a1:5a65:12::1 dev eth0 metric 1024 pref medium

```

```

root@07_K3:~# ip -6 r
2000::/64 via 2001::1234 dev eth1 metric 1024 pref medium
2001::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth0 proto kernel metric 256 pref medium
fe80::/64 dev eth1 proto kernel metric 256 pref medium
default via fd37:e7a1:5a65:12::1 dev eth0 metric 1024 pref medium
root@07_K3:~# ip -6 r
2000::/64 via 2001::1234 dev eth1 metric 1024 pref medium
2001::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth0 proto kernel metric 256 pref medium
fe80::/64 dev eth1 proto kernel metric 256 pref medium
default via fd37:e7a1:5a65:12::1 dev eth0 metric 1024 pref medium

```

3.4.1.7 Należy puścić pinga z K1 na K3/eth1/IPv6 (Podpowiedź: powinno działać)

```

root@07_K1:~# ping 2001::1
PING 2001::1(2001::1) 56 data bytes
64 bytes from 2001::1: icmp_seq=1 ttl=63 time=0.350 ms
64 bytes from 2001::1: icmp_seq=2 ttl=63 time=0.047 ms
64 bytes from 2001::1: icmp_seq=3 ttl=63 time=0.052 ms
64 bytes from 2001::1: icmp_seq=4 ttl=63 time=0.051 ms
64 bytes from 2001::1: icmp_seq=5 ttl=63 time=0.052 ms
64 bytes from 2001::1: icmp_seq=6 ttl=63 time=0.050 ms
64 bytes from 2001::1: icmp_seq=7 ttl=63 time=0.088 ms
64 bytes from 2001::1: icmp_seq=8 ttl=63 time=0.072 ms
64 bytes from 2001::1: icmp_seq=9 ttl=63 time=0.052 ms
64 bytes from 2001::1: icmp_seq=10 ttl=63 time=0.052 ms
64 bytes from 2001::1: icmp_seq=11 ttl=63 time=0.052 ms
64 bytes from 2001::1: icmp_seq=12 ttl=63 time=0.055 ms
64 bytes from 2001::1: icmp_seq=13 ttl=63 time=0.055 ms
64 bytes from 2001::1: icmp_seq=14 ttl=63 time=0.051 ms
64 bytes from 2001::1: icmp_seq=15 ttl=63 time=0.052 ms
64 bytes from 2001::1: icmp_seq=16 ttl=63 time=0.081 ms

```

3.5 Aktywne połączenia

Z K1 należy połączyć się przy pomocy SSH na K3 używając adresu K3/eth1/IPv6 (podpowiedź: w terminalu K1 wpisać: „ssh 2001::1”)

Po połączeniu należy sprawdzić listę aktywnych połączeń na K3 (podpowiedź: ss -tupn). Powinny być widoczne połączenia SSH zarówno z Komputera Studenta, jak i z K1.

3.5.1.1 Należy udokumentować listę aktywnych połączeń na K3 i zaznaczyć wpis o połączeniu z K1

```

root@07_K3:~# ss -tupn
Netid  State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
tcp    ESTAB  0        0        172.23.7.3:22       10.141.6.8:61297    users:(("sshd",pid=185,fd=4))
tcp    ESTAB  0        0        [2001::1]:22       [2000::1]:41086    users:(("sshd",pid=168,fd=4))

```

Drugi wpis informuje o połączeniu z K1

Po wykonaniu zrzutu ekranu można rozłączyć K1 z K3 wpisując w terminalu, z którego się łączono do K3 z K1, komendę „exit” – połączenie ssh zostanie zamknięte, ponownie będzie widoczny prompt komputera K1.

3.6 Kto już korzysta z IPv6?

Polecenie `nslookup` pozwala podejrzeć adresy IPv4 oraz IPv6, które są ukryte za daną nazwą domenową. Podejrzemy kilka znanych serwisów.

Podpowiedź: Należy z K1, lub innego kontenera, wpisać polecenie „nslookup <nazwa>”:

```
root@88_K1:/# nslookup facebook.com
Server:      127.0.0.11
Address:     127.0.0.11#53

Non-authoritative answer:
Name:   facebook.com
Address: 31.13.81.36
Name:   facebook.com
Address: 2a03:2880:f116:83:face:b00c:0:25de
```

Niektóre serwisy nie będą miały w wyniku adresu IPv6 (choć właściwsze byłoby sformułowanie „niektóre będą miały”).

3.6.1.1 Należy sprawdzić jakich adresów używają wybrane serwisy i uzupełnić tabelę:

Serwis	IPv4	IPv6
facebook.com	31.13.81.36	2a03:2880:f116:83:face:b00c:0:25de ☺
google.com	142.250.203.206	2a00:1450:401b:810::200e
youtube.com	142.250.203.206	2a00:1450:401b:810::200e
onet.pl	13.227.146.122 13.227.146.66 13.227.146.64 13.227.146.25	brak
gmail.com	216.58.208.197	2a00:1450:401b:800::2005
wikipedia.org	185.15.59.224	2a02:ec80:300:ed1a::1
pw.edu.pl	194.29.151.9	brak
pzkosz.pl	91.134.34.25	brak
dodo.capri.pl	217.149.242.6	2001:1a68:1::6

```
root@07_K1:~# nslookup google.com
Server:          127.0.0.11
Address:         127.0.0.11#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.203.206
Name:   google.com
Address: 2a00:1450:401b:810::200e

root@07_K1:~# nslookup youtube.com
Server:          127.0.0.11
Address:         127.0.0.11#53

Non-authoritative answer:
Name:   youtube.com
Address: 142.250.203.206
Name:   youtube.com
Address: 2a00:1450:401b:810::200e
```

3.6.1.2 Które z tych serwisów prawdopodobnie znajdują się w jednej sieci IPv6? Skąd to wiemy? Co to w praktyce oznacza?

Odp: Serwisy google.com, youtube.com oraz gmail.com prawdopodobnie znajdują się w jednej sieci IPv6, ponieważ wszystkie 3 zaczynają się prefixem 2a00:1450:401b. Najpewniej ich sieć to 2a00:1450:410b::/48. W praktyce oznacza to tyle, że wszystkie te strony są zarządzane przez jednego właściciela (sieci), w tym przypadku byłby to google, a adresy IPv6 zostały przez niego rozdzielone z dostępnej puli z tym prefixem.